Experiment No. 5

Objective: To perform data pre-processing operations i) Handling Missing data ii) Min-Max normalization.

Theory:

Data pre-processing is a critical step in data analysis and machine learning, involving the cleaning and transformation of raw data into a suitable format for analysis or modeling. It encompasses a variety of techniques and methods to enhance the quality of data, handle missing values, deal with outliers, and prepare the data for further analysis. Here is an overview of key concepts in data pre- processing:

1. Data Cleaning:

- **Handling Missing Values:**
  - Identify and handle missing values by either removing them, imputing with mean or median, or using more advanced imputation methods.
- **Outlier Detection and Treatment:**
  - Identify and handle outliers to prevent them from negatively impacting analysis or modeling results.

2. Data Transformation:

- **Normalization and Standardization:**
  - Scale numeric features to a standard range (e.g., 0 to 1) using normalization or transform them to have a mean of 0 and a standard deviation of 1 using standardization.
- **Log Transformation:**
  - Apply log transformations to handle skewed data and make it more suitable for certain analyses or modeling techniques.
- **Box-Cox Transformation:**
  - A family of power transformations that can stabilize the variance and make the data more closely approximate a normal distribution.

3. Encoding Categorical Variables:

- Convert categorical variables into numerical representations that can be used by machine learning algorithms. Common methods include one-hot encoding, label encoding, or binary encoding.

4. Handling Imbalanced Data:

- Address imbalances in class distribution for classification problems using techniques such as oversampling, undersampling, or synthetic data generation.

5. Feature Engineering:

- Create new features or modify existing ones to improve the performance of machine learning models. This can include creating interaction terms, polynomial features, or extracting information from existing features.

6. Data Reduction:

- **Principal Component Analysis (PCA):**
  - As mentioned earlier, PCA is used for dimensionality reduction by transforming data into a lower-dimensional space while retaining as much variance as possible.

7. Dealing with Duplicates:

- Identify and remove duplicate records to avoid redundancy in the dataset.

8. Handling Time Series Data:

- Apply appropriate techniques for time series data, such as resampling, smoothing, or lagging, depending on the analysis requirements.

9. Data Integration:

- Combine data from multiple sources into a unified dataset, ensuring consistency and resolving any discrepancies.

10. Data Splitting:

- Split the dataset into training and testing sets to evaluate model performance on unseen data.

11. Data Visualization:

- Visualize the distribution of data, relationships between variables, and patterns to gain insights into the dataset.

12. Handling Skewed Data:

- Address skewness in data distributions using transformations (e.g., log transformation) to make the data more suitable for certain statistical analyses or machine learning algorithms.

Effective data pre-processing is crucial for the success of any data analysis or machine learning project. It helps ensure that the data used for analysis or model training is accurate, representative, and appropriate for the intended purpose.

Code:

i) Handling Missing data

```
# Load necessary libraries
if (!requireNamespace("dplyr", quietly = TRUE)) {
  install.packages("dplyr")
}
library(dplyr)

# Function to handle missing data
handle_missing_data <- function(data, method = "mean") {
  # Display the original data
  cat("Original Data:\n")
  print(data)

  # Handling Missing Data
  if (method == "mean") {
    # Replace missing values with the mean of the respective column
    data_imputed <- data %>% mutate_all(funs(ifelse(is.na(.), mean(., na.rm =
TRUE), .)))
  } else if (method == "median") {
    # Replace missing values with the median of the respective column
    data_imputed <- data %>% mutate_all(funs(ifelse(is.na(.), median(., na.rm =
TRUE), .)))
  } else if (method == "zero") {
    # Replace missing values with zeros
    data_imputed <- data %>% replace(is.na(.), 0)
  } else {
    cat("Invalid imputation method. Supported methods: 'mean', 'median', 'zero'\n")
    return(NULL)
  }

  cat("\nData after Handling Missing Data (Imputation Method:", method, "):\n")
  print(data_imputed)

  return(data_imputed)
}

# Example data (replace this with your own dataset)
# Consider a data frame with missing values
data <- data.frame(
  A = c(1, 2, NA, 4, 5),
  B = c(5, 4, 3, 2, 1),
  C = c(10, NA, 8, 6, 4)
)

# Specify the imputation method ("mean", "median", "zero")
imputation_method <- "mean"

# Handle missing data
```

```
imputed_data <- handle_missing_data(data, imputation_method)
```

ii) Min-Max normalization

```
# Load necessary libraries
if (!requireNamespace("dplyr", quietly = TRUE)) {
  install.packages("dplyr")
}
library(dplyr)

# Function to perform Min-Max normalization
min_max_normalization <- function(data) {
  # Display the original data
  cat("Original Data:\n")
  print(data)

  # Min-Max Normalization
  min_max_normalized <- data %>% mutate_all(funs((. - min(.)) / (max(.) -
min(.))))

  cat("\nMin-Max Normalized Data:\n")
  print(min_max_normalized)

  return(min_max_normalized)
}

# Example data (replace this with your own dataset)
# Consider a data frame with numeric values
data <- data.frame(
  A = c(10, 20, 30, 40, 50),
  B = c(5, 15, 25, 35, 45),
  C = c(1, 2, 3, 4, 5)
)

# Perform Min-Max normalization
min_max_normalized_data <- min_max_normalization(data)
```